How I Stopped Losing Promises in Email Threads: The Reply Is Drafted Before I Open the Email

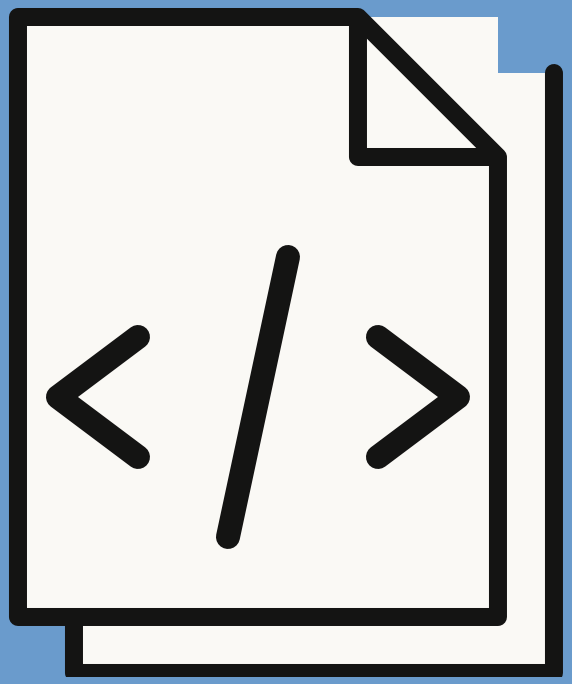
The AI files every email into a case, remembers what was promised, and drafts the reply. I check it and press Send. The problem, how it is built, and where it goes next. October 2026, updated November 2026.

A real case study by Ah Hock, The Industrial Scribe

One office in a factory in the semiconductor supply chain. Real numbers from one user over seven weeks. Anonymised: no company, customer or colleague names, no amounts, no part numbers.

Contents

Executive summary	4
The problem and the use cases	6
Architecture pattern	9
Is it effective, and where is the room to improve?	16
Looking forward	19
Next steps	21
Appendix: The architecture behind it	23
Appendix: Results in numbers	26
Appendix: Glossary	28



Chapter 1

Executive summary

Chapter 1

Executive summary

The inbox was never the problem. Starting was.

Many decisions stop with me. I am copied on about sixty percent of the mail that reaches me, which means most of it is for information, and the part that is mine to decide is buried in threads I did not start. The inbox holds around 7,400 messages. Once I reply I am short and clear, usually one to four lines. The cost was never the writing. It was the reluctance to begin, and the fact that nothing told me what was really owed on a thread before I opened it.

Three things went wrong as a result. Replies waited longer than they should. Promises made in a thread were lost when the thread went quiet. And decisions were taken twice, because the reasoning behind the first one was not written anywhere I could find it.

The Case Desk is the answer built for that. Scripts with no AI collect the mail through a bridge into the desktop mail client, file every message into a case, and measure how long each thread has waited for me. A routine runs every thirty minutes, transcribes the new mail into a ledger of facts and follow-up tasks with owners, validates every write, and tags each case by the kind of work it needs: proposal, decision, reply, blocked, chase or watch. When I say "work on" a case, a supervising session reads the decision doctrine for that customer, states which objective the reply serves, drafts it in my own voice into the Drafts folder, and checks it against the raw mail before I see it. I read, edit and press Send. Nothing is ever sent for me.

The results after four weeks: the reply is waiting before I open the case; my voice is learned from 420 of my own sent mails; the five big commitments I owe are written backwards from "done" with a chain of tasks and owners; and the trace shows the routine reads the doctrine before every draft and validates every ledger write.

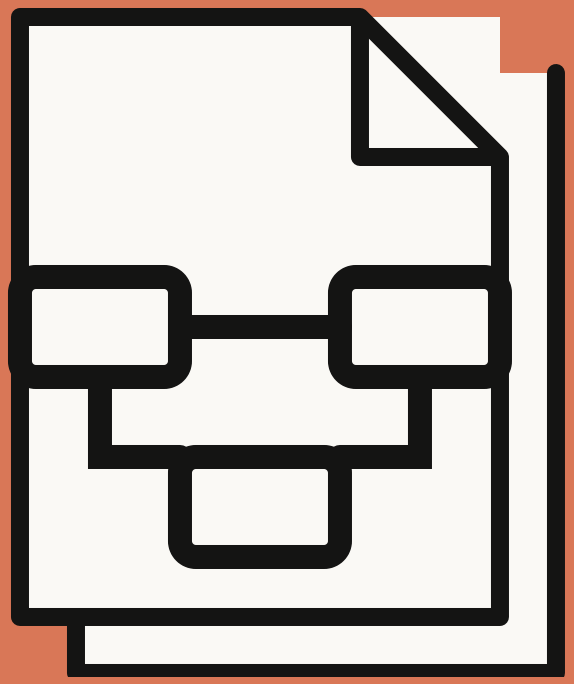
Three weeks on, the desk has grown with the work. The ledger holds about 124 cases. The doctrine covers six customers, up from three. Responsibilities have been re-split, and the desk now routes each question to the person who owns it. And the drafting has become finer-grained: it keeps my own edits, changes one phrase or one attachment in place, and continues an existing chain to a narrower audience.

The business case for an agent here

Filing mail and ageing threads are deterministic, so they are scripts. Judging what is owed on a thread, whether a request conflicts with a decision I took last month, and how to say no to a customer without telegraphing my position are not. That is where one supervising model earns its keep. It is also the only place a model is allowed to make a judgement, and the brake is absolute: nothing leaves the Drafts folder without my hand.

What this guide covers

- The problem, stated as situation, complication and resolution, and the use cases the desk serves.
- The architecture pattern: a supervised multi-agent desk on top of a script pipeline, with a worked example of one case.
- How the work flows step by step: which steps are scripts, which are the model, how it judges and what it suggests.
- The skills the AI needs, and the decision framework answered for this module.
- Whether the module is effective by the guide's six tests, with evidence, and the method for finding the room to improve.
- What going higher means for the desk, what not to build, and one next decision.



Chapter 2

The problem and the use cases

Chapter 2

The problem and the use cases

Situation

One person in the office of a factory in the semiconductor supply chain receives mail from several customer groups, from internal departments (production, QA, planning, purchasing, engineering) and from suppliers. Most of it is copy-only. Commercial replies, proposals, recovery plans and contract positions are that person's to decide; delivery, quality and service questions belong to a colleague; execution belongs to the team. The mail lives in a classic desktop mail client on a POP account, which the usual cloud mail tools cannot see.

Complication

- The hard part of a thread is rarely in the latest message. It is three replies up, or in a different thread, or in something said in a meeting.
- The barrier is at the start. Once a reply is begun it is finished quickly. Before that, a dreaded thread stays unopened.
- Promises made in mail have no owner once the thread goes quiet, and no date.
- Past decisions and their reasons lived in memory. The same question was decided twice, sometimes differently.
- Commercial replies that felt unfair stalled longest. Fairness had to be checked before wording, or the draft would sit.

Resolution

Make the case, not the message, the unit of work. Every case carries its facts, its open tasks with owners and dates, its tag, and its link to one of the five big commitments. Have the reply written before I look. Show one case at a time, smallest first when stuck. Keep every customer-facing word consistent with a written doctrine, and log every decision with its reason so it is never decided twice.

Use cases

Replies drafted before I look. The case opens with the draft already in the Drafts folder, in my voice, ending with one next action. A short holding reply ("Received with thanks, I will get back to you by Friday") counts as a win, because the barrier was starting.

The hard part in one line. Each case shows the message's hard part in one plain line, so I do not have to open a mail I am dreading to know what it asks.

Owners and dates. Every follow-up task carries one owner and one date. A rework sits with production, a purchase order with purchasing, a certificate review with QA. Follow-ups exist only under one of the five commitments; there is no flat list of overdue items.

Decisions logged, never edited. When I decide, the desk offers to append the decision, its reason and what it supersedes to that customer's log. A later request that conflicts with a logged decision is flagged before drafting.

Fairness before wording. For commercial replies, the draft states first whether the position is fair on the facts. If I think it is not, the draft says so and offers the alternative, rather than wording something I will not send.

Reply age as the measure. A script measures how long each thread has waited for my answer. It is the one number the whole module is judged by.

My edits are kept. Before touching a draft again, the desk compares it with the copy it last wrote. Anything I changed by hand stays.

Edits in place. Change one phrase, swap an attachment, or attach another mail as an item, without rewriting the draft.

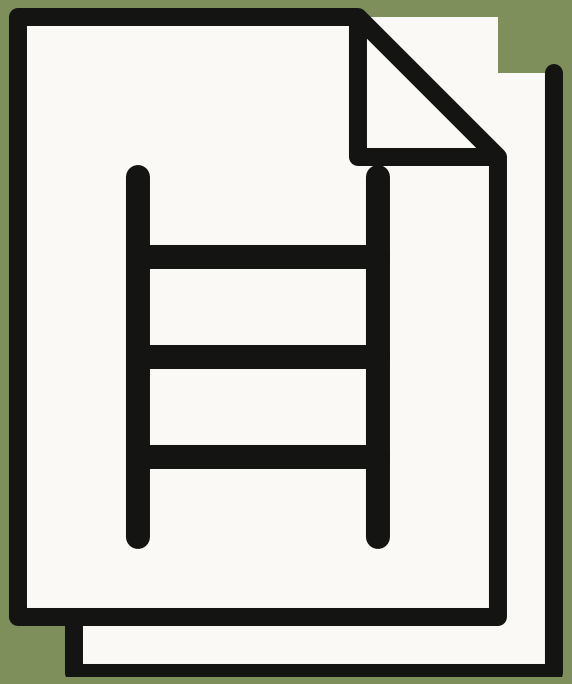
The right audience. Continue an existing chain to one person only, or forward it with its history, instead of replying to everyone.

Routed to its owner. Each question is drafted to the person who owns it; the positions that are mine stay with me.

Mismatches flagged, never merged. Two figures on different bases, a date told to a customer that does not match the plant schedule, a claim QA has not cleared: each is raised before drafting, never quietly reconciled.

Handover packs. Source mails saved as files, grouped by account and zipped, so a case can be handed to someone else with its evidence.

Work carried across sessions. The desk reads the record of other working sessions, so a deck finished in a separate contract chat is picked up rather than rebuilt.



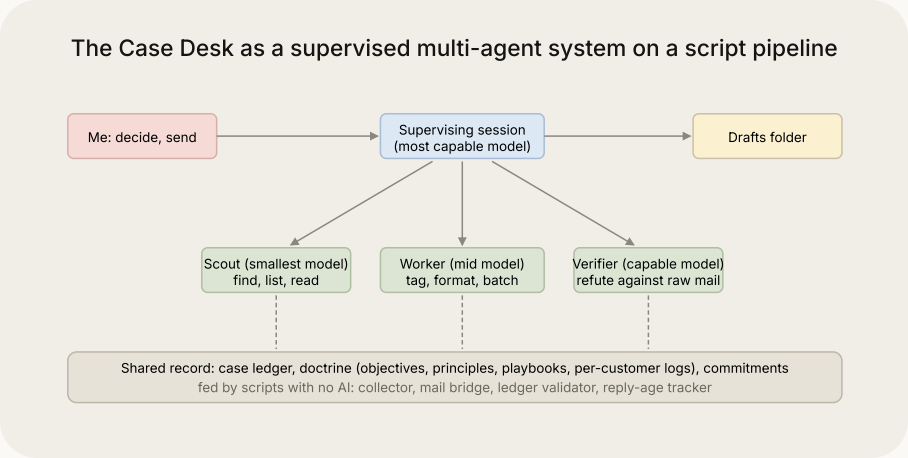
Chapter 3

Architecture pattern

Chapter 3

Architecture pattern

Hierarchical, supervisory: one judge, cheap specialists, a shared record



Architecture overview: a sequential script pipeline feeds a shared record; one supervising session makes every judgement and delegates to three specialist roles as if they were tools; the specialists are cheaper models for cheaper work; everything reads and writes the same record. The supervisor never hands its conversation with me to a specialist. The guide calls this full orchestration rather than routing.

When to use: several unrelated domains arrive together (a rework for one customer, a contract clause for another, a renewal for a third), rules must be enforced centrally, and control is moderate for routine replies but high for anything commercial. The supervisor enforces the doctrine; the specialists never need to know it.

When to avoid: for one-line replies and filing. Those never trigger a specialist. The routine copies mail packs itself rather than spawning a worker, a decision taken on 12 October after the trace showed the worker added nothing.

How it works, step by step

There are three ways work enters the desk: on a timer, when I ask for a case, and when I tell it something that never reached email. In each one the same division holds. A script does everything that has one right answer. The model does only the reading, the judging and the wording. I decide and send.

Flow 1: the thirty-minute routine (weekdays, 08:00 to 18:00)

Step	Who does it	What happens	What it produces
1. Collect	Python script, no AI, about 30 seconds	Exports new inbox and sent mail through the bridge. Files each mail to a case by that case's rules: keywords, part numbers, sender domains, named people. Then widens: any mail in the same conversation as one already filed, and any mail from or to that case's people. Finds mails that name me and checks whether I have answered on that thread since.	One reading pack per case that has unseen mail; a list of mails naming me that still need a reply; a "mail window" the validator will use
2. Extract	The model (the routine itself)	Reads each pack and writes operations, not prose: one event per meaningful mail in thirty words or fewer with exact part numbers, dates and quantities; a new follow-up for every new obligation, with owner, direction (we owe or they owe) and due date; a closure for every open follow-up that the new mail satisfies; a changed fact when the position moved; a changed status when urgency moved.	An operations file
3. Validate	Python script	Checks every operation: the mail it cites must exist in the mail window, the case must exist, dates must parse, every value must be one of the allowed ones. It must print "all OK" or nothing is applied.	A pass, or a list of errors to fix
4. Apply	Python script (the ledger tool)	Writes the operations into one file per case. Follow-ups get stable numbers. Nobody, model or person, edits the ledger by hand.	The updated ledger: rules, facts, follow-ups, timeline
5. Brief and tag	The model	Rewrites the case's stored brief (position in 25 words, advice in 45, next step in 12) and its work tag: proposal, decision, reply, chase, watch or blocked; how soon; how much effort; what the deliverable is.	What the desk shows for that case
6. Mark read	Python script	Clears the unread flag only on mails the desk has processed.	A mailbox where "unread" means "not yet seen by the desk"
7. Age	Python script	For every thread I am on, counts the days since the latest inbound mail, unless I have replied or told the desk about it since.	The reply-age list
8. Alert	Python script	A desktop notification when a mail that names me needs a reply, or when drafts are waiting.	One notification. Nothing is sent.

Steps 1, 3, 4, 6, 7 and 8 are scripts. The model is used in two places only:
turning mail into structured facts, and writing the brief.

Flow 2: "work on" a case, in the chat

Step	Who does it	What happens	What I see
1. Find the case	Script	Resolves what I typed to a case, tolerant of typos. If the match is weak it asks which case; it never guesses silently.	The case, or one question
2. Read the doctrine	The model	In order: the shared objectives and principles; then that customer's folder (profile, objectives, principles, playbook, live state, decisions log); then the communication playbook.	One line: which customer, which objective this reply serves
3. Read the evidence	The model, through scripts	The ledger for that case; the underlying thread and my own sent replies through the bridge; attachments that carry the substance (minutes, letters, schedules); shared drives, read-only; an order or stock fact through the ERP guide when the case needs it.	Nothing yet
4. Judge	The model	What is really owed, and by whom. How urgent, on evidence. Whether email and what I said disagree. Whether the position is fair on the facts. Whether the request conflicts with an objective, a principle or a logged decision.	The hard part in one line, and any conflict flagged for me to choose
5. Suggest	The model	Lays out the options with a recommendation and its reasoning. For a proposal: the ask in one line, the three to five inputs needed, which it already has from mail, and a request for the first missing one only.	A recommendation, or one question
6. Draft	The model writes; a script files it	The wording is written in my voice from the tone profile and the wording rules. The bridge creates it as a reply on the right thread, to the right audience, in the Drafts folder. If I edited an earlier version, my edits are kept.	A draft waiting in Drafts
7. Verify	A second model (the verifier)	For anything significant, tries to refute the draft against the raw mail: every date, quantity and part number.	Corrections, if any
8. Report and log	The model	Three points at most and one next action. If I decided something, it offers to append the decision, the reason and what it supersedes to that customer's log.	The reply in chat; I press Send in the mail client

Flow 3: something I tell it that never reached email

I type a line such as "this case: spoke to the buyer, hold until next month, revised order by Friday". The desk resolves the case, logs it as a spoken update (who I spoke to, what was agreed, in my words), changes the facts it affects, opens a follow-up for each commitment and closes any it settles. It then reads back in three to five lines what it logged, and flags any conflict with the email trail, for example a date promised in writing that I have now moved. The old value is kept, never overwritten. If a commitment to or from a customer has no written trace, it says so and offers a short confirming mail.

How it judges

Judgement is where a model is weakest, so the rules are written down. They came from a check in September: on a sample, the facts it extracted were exact fifteen times out of fifteen, but ownership and urgency were wrong about fifteen percent of the time. The rules that followed:

- **Owner is me only when** I am on the "To" line and I am named in the body, or I am the only addressee, or I made the commitment myself. If a colleague picked the thread up in later mails, the owner is that colleague, even if the first mail came to me. If I was only copied, it says so.

- **A question is not a deadline.** "Could you quote by the 11th?" has no due date unless someone confirmed it. "Response by the 18th" does.
- **Escalation needs evidence:** a chaser, a senior person added to the thread, a missed committed date, or threat language. One routine mail with a future date is simply open.
- **Read the cautions first.** Before telling me "we owe this", it reads the case's caution and blocker facts. It never pushes me to send something the thread shows cannot be sent yet.
- **Closing beats adding.** Before adding a follow-up from new mail, it checks whether that mail satisfies an open one.
- **Email says, or I said.** Every fact is tagged by where it came from, and the two are never blended.

What a case looks like when it reaches me

No more than about eight lines:

1. **Position** in one sentence, hard part first.
2. **New** since last time: who said what, three bullets at most, exact part numbers, dates and amounts.
3. **I owe** (with due date) / **the team owes** (by name) / **they owe** (by name, days waiting).
4. **Risk or conflict** in one line.
5. **Proposed next step**, and whether a draft is already waiting.

Wording rules learned in practice

These were added one correction at a time and now apply to every customer draft:

- Never apologise. "Thanks for your patience" replaces "sorry for the delay".
- Positive wording that keeps a firm condition: "we will arrange the deliveries upon receipt of the revised order", not "we will have difficulty delivering".
- Put a response date in the record, so a delay sits with whoever owes the answer.
- No internal cost, margin or supplier names in customer material.
- No self-justifying lines, no priority grandstanding; keep my own wording and tidy the grammar only.
- Holds, prices, recovery plans and dates are my call, never the desk's.

Key challenge: context management

The supervisor's context fills with mail. The answer is a file-based memory outside the context window: the ledger holds the facts, the doctrine holds the rules, a memory folder holds how I like to work, and the trace keeps fourteen days. Packs are transcribed into structured operations and validated before they count, so the supervisor reads state rather than raw threads. The bridge returns subjects, dates and counts before bodies, which is the guide's advice to cap tool responses and page through results.

Key challenge: the mail client's guard

The desktop mail client's programmatic-access guard blocks sender addresses, bodies and sending from code, and hangs the call waiting for a modal "Allow" prompt. The fix is a trust setting that only the user may change, and the bridge carries a watchdog and a guard check so a returning prompt fails fast instead of freezing the routine. The live mail store is pinned by name; the default store is a test account and must never be used as a fallback.

Design best practices, applied

Choose the right model for the job. Scripts for the deterministic (collect, file, validate, age). The smallest model for lookups, the mid model for mechanical edits, the most capable model for judgement and for adversarial checking. Proposals, prices, recovery plans and negotiation positions are never delegated at all.

Practice modular design. Prompts live in central files: the project instructions, each skill, the doctrine. Tools are discrete scripts. Agents are roles that take only the tools they need. A new customer is a new folder from a template; a new rule is a line in a playbook.

Extend capabilities with skills. A skill is a packaged file of know-how the model loads only when the task matches. The guide treats skills as the first way to grow an agent, before adding more agents. The table in the next section lists the ones this module needs.

Build observable systems. Every run is traced: which model, which command, whether the validator ran, whether the doctrine was read before drafting, and whether any send command appeared anywhere, including inside a specialist.

Skills the AI needs for this module

Skill	What it holds	When it loads	Today
Decision doctrine	Ranked objectives, principles, playbooks, and a folder per customer (profile, live state, append-only decisions log). Six customers covered.	Before any customer wording or advice	Exists
Email routine	The daily routine, escalation rules and hard rules: draft only, never send; the bridge is the only code that touches the mail client	"What's today", "check my mail", "reply to"	Exists
ERP guide	Screens, exports and click paths; what order, stock and purchase data mean	When a case needs an order, stock or purchase fact	Exists
Costing method	How a price is built for one customer, with its cross-check pack. Never compared across customers	Proposal cases that carry a price	Exists
Presentation playbook	Chart choice, slide structure, a pre-send checklist	When the case's deliverable is a deck	Exists
Browser reading	Reading a shared sheet or page the user sends	When a link arrives in a case	Exists
Wording rules	The five rules above, plus the drafting rules (no self-justifying lines, no priority grandstanding)	Every customer draft	Partly: spread across notes and the communication playbook, not yet one skill
Draft checker	Scoring criteria for a draft against the wording rules and the fairness check	Before a draft is shown	Missing
Chase on due date	How to chase an owner when a follow-up task falls due	When a task's date arrives	Missing

In one long working session the desk invoked four of these skills (doctrine, ERP guide, presentation playbook, browser reading), ran about 1,900 commands, most of them through the mail bridge, and started 24 specialist runs.

Decision framework for this module

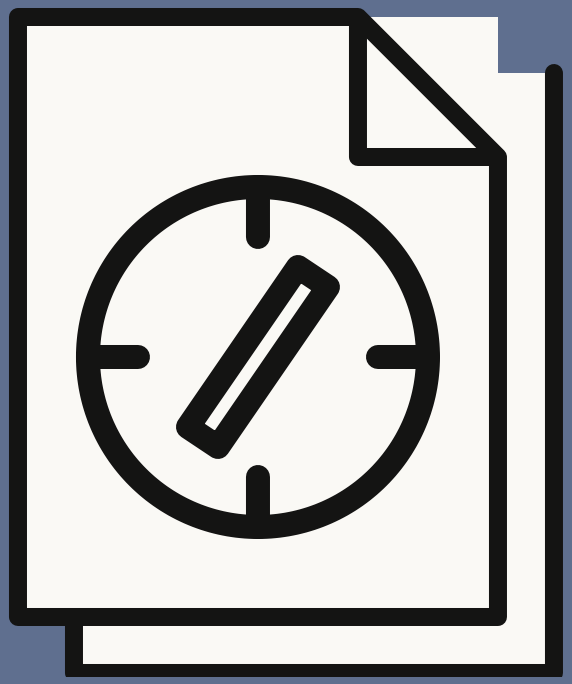
Control: high for anything commercial or dated, so those stay in the supervisor and then with me. Moderate for routine replies, so a supervised desk fits.

Domain complexity: multi-domain and unpredictable per thread. A single generalist agent would fall off; a supervisor with a doctrine does not.

Resources: one laptop, a routine every thirty minutes. Specialists are used sparingly; the routine should wake a model only when there is new mail.

Expertise: deep but single-domain per task, so it lives in skills (doctrine, costing method, communication tone), not in more agents.

Step on the ladder: step 3, one supervisor delegating to specialists, with the brakes at the two high-control points: judgement stays in one session, sending stays with me. Put another way: inside one task it is an agent, choosing its tools, running several steps and checking the result. Across the day it is a human-approved workflow, because every outward action stops at the Drafts folder. The thirty-minute routine is the only scheduled autonomous part, and it only files and triages.



Chapter 4

Is it effective, and where is the room to improve?

Chapter 4

Is it effective, and where is the room to improve?

The guide never defines an effective agent in one sentence. It defines what an agent is, a model that directs its own steps and tools toward a goal, and then lets "effective" emerge from six tests scattered across its chapters. This chapter applies those six tests to the Case Desk with evidence, then shows the method used to look for the room to improve, so the search can be repeated next quarter without this document.

The six tests, with evidence

Test from the guide	What it asks	Evidence on the Case Desk	Verdict
1. Right problem for an agent	Is the path unknown in advance, so a script could not do it?	Judging what is owed on a thread, spotting a conflict with a past decision, and wording a refusal without telegraphing position cannot be scripted. Filing, ageing and validating can, and are scripts.	Pass
2. Simplest design that works	Started small, added only on measured need?	Began as one agent reading mail (15 September); specialists added three days later when work types diverged; the worker was removed from the routine on 12 October when the trace showed it added nothing.	Pass
3. Matched to control, domain, budget, expertise	The four decision questions answered honestly?	High-control work (proposals, prices, sending) never delegated; multi-domain threads handled by a supervisor; specialists used sparingly; expertise in skills.	Pass
4. Observable	Can we see why it decided, not just what?	Every run traced: model, commands, validator ran, doctrine read before drafting, no send command anywhere. A sample check in September measured the output itself: facts exact 15 of 15, ownership and urgency wrong about 15 percent, which became the written judgement rules.	Pass
5. Modular	Can parts change without a redesign?	Prompts in central files, tools as discrete scripts, agents as roles, customers as folders from a template.	Pass
6. Metric tied to a business outcome	Is there one number, and is it moving?	Reply age is measured per thread. The trend over the four weeks is not yet reported on the desk as a single line.	Partial

Why it counts as effective. Five of six tests pass on evidence, and the sixth has the measure in place but not yet the trend. The strongest argument is test 1 together with test 3: the desk puts a model only where a script cannot go, and keeps it out of every decision a customer will hold me to. The guide's view is that an agent nobody can audit is not effective in a high-control job however clever it is; this one can be audited run by run.

How to look for the room to improve

The guide gives no "improvement" chapter either, but it gives six levers, and each one is a place to look. Applied in order:

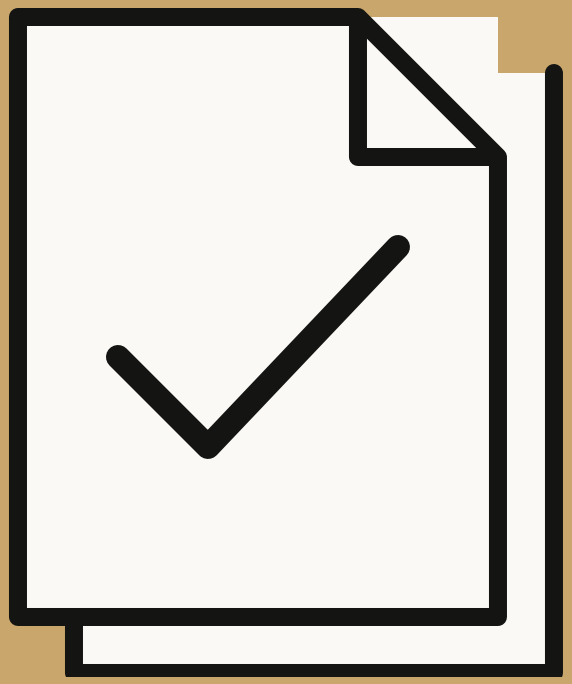
1. **Re-ask the four decision questions.** Has control, domain, budget or expertise changed since the design? Here: the domain widened from "reply to mail" to "keep delivery commitments", which the current desk only partly serves.

2. **Read the pattern's "when to avoid" list against actual use.** The guide says avoid a single generalist for multiple domains, avoid parallel work where there is no rule for contradictions, avoid draft-and-check where criteria are subjective. The desk's criteria for a good draft are written, so the absence of draft-and-check is a gap, not a choice.
3. **Compare the map with the trace.** Steps the map has that never ran, runs the map does not describe, rules skipped. The idle worker was found this way; the next candidates are routine runs that woke a model with no new mail. A second gap shows in the numbers: the design names three specialist roles, but of 24 specialist runs in one long session, 16 were general-purpose and only 3 were the worker, 2 the verifier and 1 the scout. Either the roles are too narrow for the real work, or the supervisor is not reaching for them.
4. **Watch the metric.** A metric that is measured but not trended is a blind spot. Reply age needs a weekly line.
5. **Reopen every key-challenge box.** Each box is a known gap with a named fix. Context management is answered; the mail client's guard is answered; there is no box yet for "promises made outside mail".
6. **Check the ladder before climbing it.** Is a step-2 pattern missing (sequential chase on due date, parallel at-risk check, draft-and-check) before any thought of more agents? All three are missing, and all three are cheaper than a new agent.
7. **Check reach.** What can the tools not see? Promises made on a messaging app or in person, and supplier dates in the ERP. An agent cannot coordinate what it cannot reach.

The room to improve, found by that method

Lever	Finding	Size
Decision questions	The desk serves replies; the real problem is the commitment chain	Large
When to avoid	Draft-and-check absent although criteria are written	Medium
Map versus trace	Routine wakes a model on empty cycles	Small
Map versus trace	Designed roles (scout, worker, verifier) took 6 of 24 specialist runs; 16 were general-purpose	Medium
Ladder (skills first)	Wording rules are not yet one skill; draft checker and chase-on-due skills are missing	Medium
Metric	Reply age measured, not trended	Small
Key challenges	No box for promises made outside mail	Medium
Ladder	Chase-on-due, at-risk check, draft-and-check all missing at step 2	Medium
Reach	Messaging-app promises and supplier dates invisible	Large

Chapter 5 orders these into what to build next.



Chapter 5

Looking forward

Chapter 5

Looking forward

What going higher means for the desk

1. **Draft-and-check before I see a draft.** The guide's evaluator-optimizer pattern, best suited to professional communication where tone matters. The criteria already exist in writing: the drafting rules, the communication playbook, the fairness check. A second call scores every draft against them and the first revises, two cycles at most. Today the verifier checks facts, not tone.
2. **Chase from the date, not from my prompt.** When a follow-up task falls due, the routine drafts the chase to its owner in my voice; I send; the reply is read back and the task updated.
3. **A live commitment record.** The five commitments and their chains of tasks become structured links with owner, due date, source of proof and last-confirmed date, replacing the hand-written file. The desk then reports commitment status, not case counts.
4. **Wake on change.** The routine runs every thirty minutes whether or not mail arrived. A script guard should wake the model only when there is something new.

What not to do. Never automate sending; the brake is the design. Do not climb to peer-to-peer agents; the control need is high. Do not show me the pile; one case at a time is a rule, not a preference.

Measure of success. Reply age going down, and follow-up tasks closed on their date.



Chapter 6

Next steps

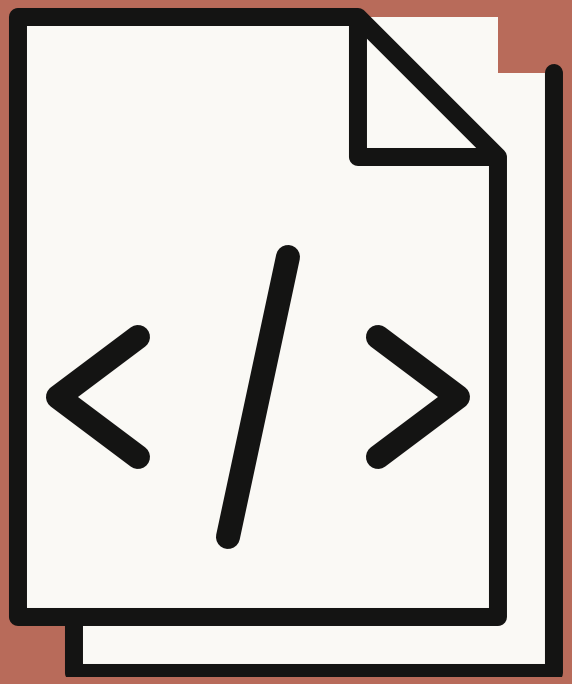
Chapter 6

Next steps

Decision, yes or no: run the draft-and-check loop on every "reply" and "chase" draft for two weeks, scoring against the written drafting rules, and compare how many drafts I edit before sending against the two weeks before.

If you want a desk like this

You don't need to rebuild it from scratch. The Industrial Scribe is packaging this design as a kit, the Case Desk: every customer thread turned into a case, a ledger of what is owed, by whom and by when, a folder per customer with a decision log, replies drafted in your own voice, and a hard rule that nothing ever sends itself. It starts with pasting a thread, works with any email system, and will be listed in the shop at theindustrialscribe.com.



Appendix

Appendix: The architecture behind it

Appendix: The architecture behind it

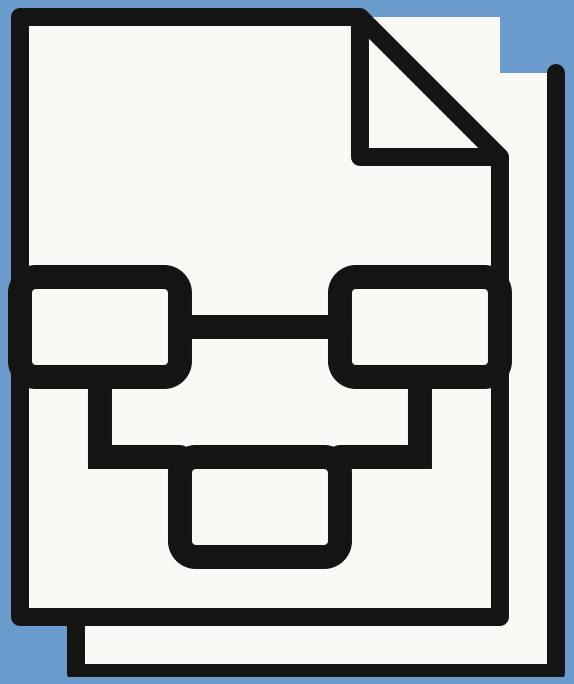
Layer	Component	What it does	AI?
Input	Mail bridge script	Reads my own mailbox through the desktop mail client already open on my laptop: scan, read, thread, attachments, save, create and edit drafts. It never sends unless I say so.	No
Input	Collector	Files each message into a case by rules: keywords, part numbers, people, domains	No
Input	Reply-age tracker	Measures how long each thread has waited for me	No
Memory	Case ledger	One file per case: facts with history, follow-ups (owner, direction, due date), timeline, brief, work tag	Written by the routine, read by all
Memory	Doctrine	Ranked objectives, principles, big commitments, decision log; one folder per customer; shared playbooks	Written by me, read before every draft
Memory	Persistent notes	About 37 short notes on how I want things done, each from a correction	Loaded every session
Processing	30-minute routine	Turns new mail into ledger operations, runs the validator, writes the brief and the work tag	Yes
Processing	Main session	Plans, judges, drafts, reviews, talks to me	Yes
Processing	Specialists	Scout (lookups), worker (mechanical edits), verifier (tries to refute a result)	Yes
Output	Drafts folder	The only place a message can go	n/a
Output	Case Desk page	A local web page where I study cases one at a time	No

Which model does what. The most capable model runs the main session and the verifier, because judgement, wording and refutation are where mistakes cost most. A mid-tier model does mechanical edits. The fastest model does broad, read-only lookups. The routine uses the app default, because every write it makes is checked by a script.

Boundaries. Everything runs on the company laptop and network. Shared drives are read-only. The assistant never enters a password, creates an account or changes a security setting. Every outward action stops at Drafts.

The guardrails, and where they came from. Each rule below came from one of my own corrections and is stored as a persistent note.

Rule	Why
Draft only, never send	I send every message; outward actions can't be undone
Employer data stays on the company network	Governance, and less to leak
The user decides; the assistant advises	Options, one recommendation, then a yes/no
Doctrine first	One line naming the counterparty and the objective before any draft
Flag conflicts, never resolve them silently	"This contradicts the decision logged on..."
Promise only what capacity, material and date support	Dates stay in brackets until the plant confirms them
Check fairness before wording a commercial reply	A reply that feels unfair is the hardest to start



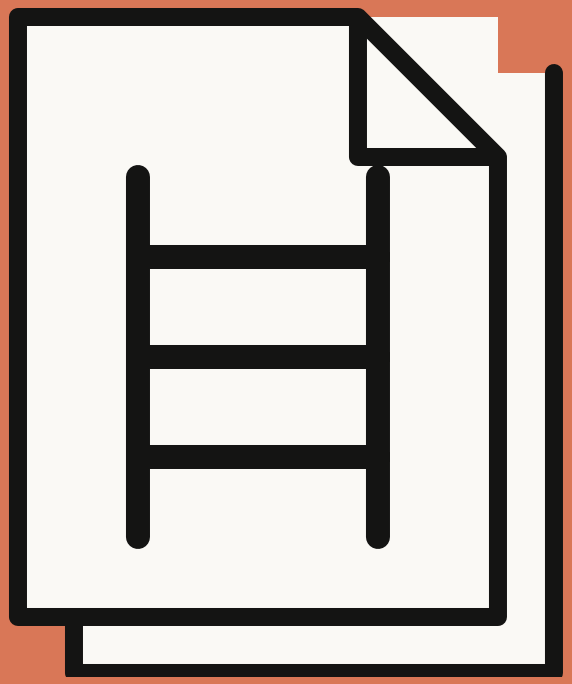
Appendix

Appendix: Results in numbers

Appendix: Results in numbers

Measured on 6 to 9 November 2026 from the assistant's own ledger, draft files and session logs. Counts only. This is a real system run by one person (me) in a real factory office since 15 September 2026, not a demo. The numbers show what the design can do for one user over seven weeks, not what every office will get. Where it fell short, this guide says so.

Measure	Value
Cases tracked (17 Sep to 6 Nov 2026, seven weeks)	124
Follow-ups tracked	1,040 (827 owed by my side, 213 owed to me)
Follow-ups closed	583 (56%), median 2 days from opened to closed, 289 within one day
Facts held, each with its history	851
Timeline events	3,236 (2,744 inbound mails filed, 230 outbound, 91 spoken updates, 154 notes)
Routine runs (every 30 minutes, weekdays)	420, processing 2,692 case packs and 691 validated write batches
Drafts written for me to send	291
My own sent mails used to learn my voice	420
Judgement check (September sample)	Facts exact in 15 of 15 cases; ownership and urgency wrong about 15% of the time, which led to the written judgement rules



Appendix

Appendix: Glossary

Appendix: Glossary

Case. One thread or topic with its facts, open tasks, owners, dates and tag. The unit of work on the desk.

Tag. The kind of work a case needs: proposal, decision, reply, blocked, chase or watch.

Ledger. The structured record of facts and tasks per case, written only through validated operations.

Doctrine. The written decision framework: ranked objectives, principles, playbooks, and per-customer folders with a live state and an append-only decisions log.

Commitment. One of the few big promises I owe, written backwards from "done" as a chain of tasks with owners.

Bridge. The only code allowed to read or write the mail client, through its local automation interface.

Supervisor, scout, worker, verifier. The judging session and its three specialist roles: cheap lookup, mechanical edits, adversarial checking.

Reply age. How long a thread has waited for my answer. The module's one measure.

Skill. A packaged file of know-how the model loads only when the task matches, such as the doctrine or the ERP guide.

Wording rules. The standing rules for customer drafts: never apologise, positive wording with a firm condition, a response date on record, no internal numbers.

Draft-and-check. One call writes, a second scores against set criteria, the first revises. The guide's evaluator-optimizer.